SIGMOID

AI Acceleration Engineer - Internal Posting

Team	AIA (AI Acceleration) Team
Positions	4

Team: AIA (AI Acceleration) Team

Positions: 4

The Role

You're an AI-accelerated builder. The work is to take an ambiguous problem and ship a working system against it - code that runs, tests that pass, documentation a teammate can follow - using Claude Code and the surrounding toolchain to compress what used to take weeks into days.

You think and build across practices: Data Engineering, Data Science, DevOps, Product, and Data Analytics. Not a specialist in any one - a generalist who can pick up a problem in any of them, choose a credible direction, and ship against it. The mental model is closer to a Forward Deployed Engineer than to a traditional vertical engineer.

What Makes This Different

Standard Engineering Role	AI Acceleration Engineer
Works on one project for months	Rotates across projects every 4-6 weeks
Deeply embedded in one domain	Fluent across DE, DS, DevOps, and Product
Writes all code manually	Uses Claude Code within a structured methodology
Owns the code long-term	Hands off after 4-6 weeks with structured overlap pairing
Success = feature delivered	Success = project team builds on the skeleton (>70% adoption)

The core skill is judgment, not speed. The AIA engineer knows when to trust the AI, when to override it, and when to write from scratch.

Responsibilities

During Engagements (4-6 weeks per project):

- Build features against the skeleton architecture set by the AIA Lead; surface implementation-level trade-offs as inputs to ADRs
- Build the functional code foundation: Claude Code for scaffolding and refactoring, manual coding for complex logic
- Generate tests alongside code; achieve >60% coverage on skeleton code
- Produce handoff-ready documentation: README, ADRs, production-readiness manifest, extension guide
- Follow client/project coding standards, not personal preferences

During Handoff (2-3 days):

- Day 1: Participate in the AIA Lead's architecture walkthrough; answer questions on features you built
- Day 2: Pair programming - each receiving engineer makes at least one code change
- Day 3: Independence verification - receiving team works solo, you're on standby

Between Engagements:

- Contribute to reusable patterns, templates, and internal tooling
- Document proven prompt patterns and engagement learnings for future pods

---

Required Skills

AI-Assisted Coding

- Hands-on experience with Claude Code for project scaffolding, multi-file generation, and refactoring
- Iterative AI loop discipline: generate → review → refactor cycles visible in your git history; not prompt-and-dump
- Trust-but-verify discipline: you review every AI-generated block before committing

Architectural Judgment

- Implement against the architecture set by the AIA Lead - understand it deeply enough to build with it, not around it
- Pattern consistency across the codebase; detect and correct AI drift
- Sharp ADR contributions: surface implementation-level trade-offs the AIA Lead needs to document; participate in architecture decisions as a builder

End-to-End Delivery

- Comfortable working across the full delivery stack - data pipelines, ML/AI, APIs, frontend, infrastructure - not a specialist in one but effective across all
- Quick to ramp into unfamiliar domains; AI tools accelerate the curve but curiosity and initiative drive it

- Comfortable with Python, JavaScript/TypeScript, and adapting to client tech stacks

Delivery & Communication

- 2+ years in client-facing delivery; comfortable with compressed 4-6 week timelines
- Testing and quality ownership - write tests alongside code, not after
- Crisp async communication: daily updates, clear blockers, no surprises for stakeholders
- Scope discipline - deliver what was agreed
- Experience handing off code to another team

What We Won't Accept

- Prompt-and-dump - accepting AI output without review or iteration
- Cargo-cult ADRs - boilerplate decision records with no real trade-off captured
- Going deep in one domain at the expense of the others
- Treating tests as paperwork - happy-path-only after the fact
- Re-architecting on handoff day instead of during build

Evaluation Process

1. **Vibe Coding Evaluation Project (4-6 hours, asynchronous):** given a realistic client problem statement, build an end-to-end skeleton covering data foundation, predictive intelligence with LLM, API, UI, and production-readiness
2. **Submit three deliverables:** Git repository, video demo (5-10 minutes), and a methodology spreadsheet documenting your AI tool usage, time allocation, and decisions. Fully offline; no interview required.